In [11]:
```
#1 Import libraries and load dataset

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import confusion_matrix, classification_report

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Input, Dense

df = pd.read_csv("ecg.csv", header=None)
```

In [12]:
```
#2 Separate features and labels

X = df.iloc[:, :-1].values
y = df.iloc[:, -1].values

# Convert labels to integers if needed
y = y.astype(int)

scaler = MinMaxScaler()
X_scaled = scaler.fit_transform(X)

# Assumes 0 = normal and 1 = abnormal
X_normal = X_scaled[y == 0]
```

In [13]:
```
#3 Split normal data for training

split = int(0.8 * len(X_normal))
x_train = X_normal[:split]
```

In [14]:
```
#4 Build and train Autoencoder

model = Sequential([
    Input(shape=(x_train.shape[1],)),
    Dense(32, activation="relu"),
```

```
    Dense(16, activation="relu"),
    Dense(32, activation="relu"),
    Dense(x_train.shape[1], activation="sigmoid")
])

model.compile(optimizer="adam", loss="mse")

model.fit(
    x_train, x_train,
    epochs=10,
    batch_size=32,
    validation_split=0.1
)
```

```
WARNING:tensorflow:TensorFlow GPU support is not available on native Windows for TensorFlow >= 2.11. Even if CUDA/cuDNN are ins
talled, GPU will not be used. Please use WSL2 or the TensorFlow-DirectML plugin.
Epoch 1/10
47/47 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - loss: 0.0178 - val_loss: 0.0060
Epoch 2/10
47/47 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 0.0066 - val_loss: 0.0047
Epoch 3/10
47/47 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 0.0059 - val_loss: 0.0044
Epoch 4/10
47/47 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 0.0053 - val_loss: 0.0038
Epoch 5/10
47/47 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 0.0046 - val_loss: 0.0031
Epoch 6/10
47/47 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 0.0039 - val_loss: 0.0028
Epoch 7/10
47/47 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 0.0034 - val_loss: 0.0025
Epoch 8/10
47/47 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 0.0031 - val_loss: 0.0024
Epoch 9/10
47/47 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 0.0028 - val_loss: 0.0022
Epoch 10/10
47/47 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 0.0026 - val_loss: 0.0020
```

Out[14]: <keras.src.callbacks.history.History at 0x1fd259a4c20>

In [15]:
```
#5 Calculate anomaly threshold
```

```
train_pred = model.predict(x_train, verbose=0)
train_error = np.mean(np.square(x_train - train_pred), axis=1)

threshold = train_error.mean() + 3 * train_error.std()
print("Anomaly Threshold:", threshold)
```

```
Anomaly Threshold: 0.014457701221298906
```

In [17]:
```
#6 Detect anomalies

test_pred = model.predict(X_scaled, verbose=0)
test_error = np.mean(np.square(X_scaled - test_pred), axis=1)

y_pred = (test_error > threshold).astype(int)

print("Confusion Matrix:")
print(confusion_matrix(y, y_pred))

print("Classification Report:")
print(classification_report(y, y_pred, zero_division=0))
```

```
Confusion Matrix:
[[2027   52]
 [2813  106]]
Classification Report:
              precision    recall  f1-score   support

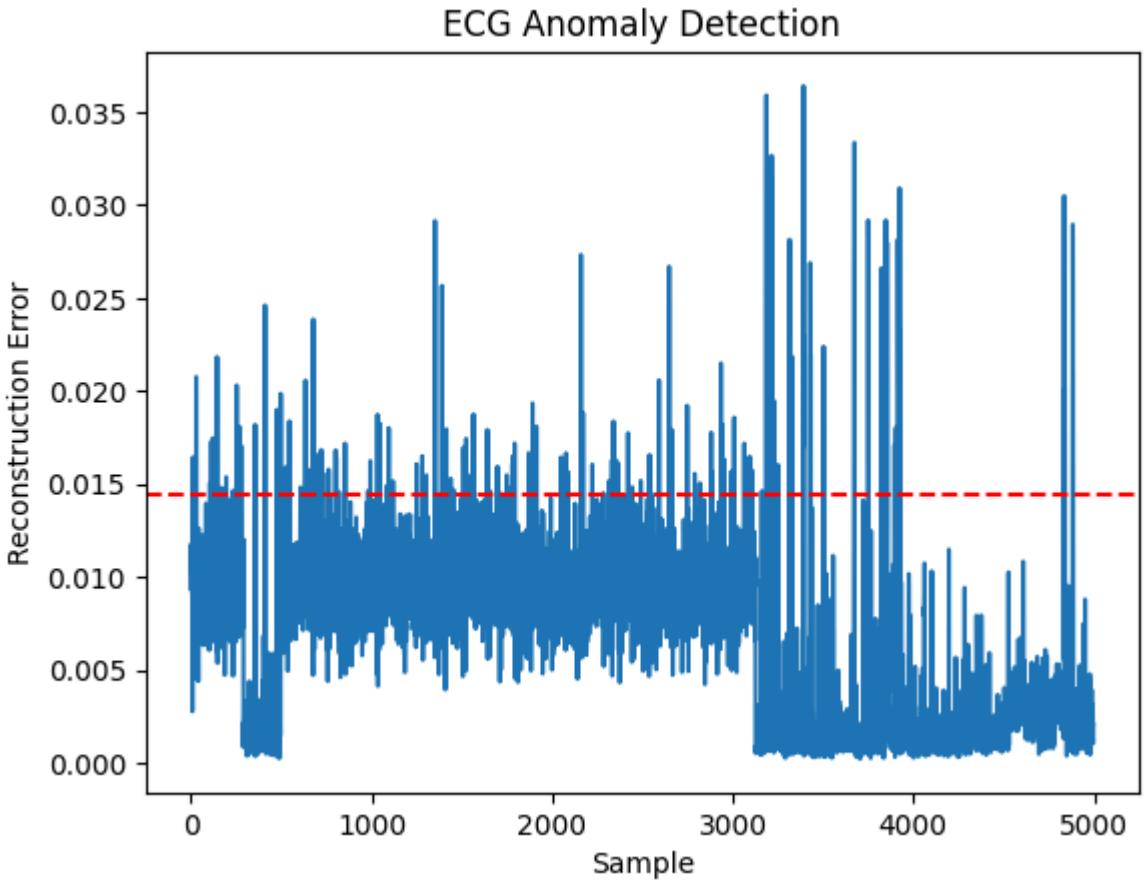
           0       0.42      0.97      0.59      2079
           1       0.67      0.04      0.07      2919

    accuracy                           0.43      4998
   macro avg       0.54      0.51      0.33      4998
weighted avg       0.57      0.43      0.28      4998
```

In [18]:
```
#7 Plot reconstruction error

plt.plot(test_error)
plt.axhline(threshold, color="red", linestyle="--")
plt.xlabel("Sample")
plt.ylabel("Reconstruction Error")
```

```
plt.title("ECG Anomaly Detection")
plt.show()
```



In [ ]: